

Lab Assignment -20

Name: D.Bharath kumar

HTNo: 2303A51097

Batch: 02

Lab 20 – Security Testing: Identifying Vulnerabilities in AI- Generated Code

Lab Objectives:

- Understand how to test AI-generated code for common security vulnerabilities.
- Learn to apply secure coding principles while analyzing AI outputs.
- Practice detecting risks such as SQL injection, XSS, hardcoded credentials, and weak encryption.
- Enhance code reliability and safety by using AI for secure refactoring.

Task 1 – Password Storage Vulnerability Check

Task: Analyze an AI-generated signup script for insecure password storage.

Instructions:

- Prompt AI to generate a Python user registration program.
- Check whether passwords are stored in plain text.
- Improve the script using hashing (bcrypt or hashlib).

Expected Output:

- A secure registration script with hashed password storage.

```

1 #generate a Python user registration program. Check whether passwords are stored in plain text. Improve the script using hashing (bcrypt or hashlib).
2 import bcrypt
3 def register_user(username, password):
4     # Hash the password using bcrypt
5     hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
6
7     # Store the username and hashed password in a dictionary (simulating a database)
8     user_data = {
9         'username': username,
10        'password': hashed_password
11    }
12
13    return user_data
14 # Example usage
15 username = input("Enter your username: ")
16 password = input("Enter your password: ")
17 user_info = register_user(username, password)
18 print("User registered successfully!")
19 print("Stored user data:", user_info)
20 # Note: In a real application, you would store the user data in a secure database rather than printing it out.
21
22

```

Output :

```

/OneDrive/Pictures/Desktop/AIAC/assg_20.py
Enter your username: bharath
Enter your password: B@123
Username: bharath
Hashed Password: $2b$12$1Lwg5zYosBaCF3hjKkP9wuvzkdF4bUELuV5rTCTQj1hqCYQghib5C
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>

```

Task 2 – Hardcoded Secrets Detection

Task: Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

Expected Output:

- A secure script with secrets managed properly.

```

23 #generate a script that connects to an external API. Identify if API keys/passwords are written directly in code. Refactor using environment variables or c
24 import os
25 import requests
26 def get_api_data():
27     # Load API key from environment variable
28     api_key = os.getenv('API_KEY')
29     if not api_key:
30         raise ValueError("API_KEY environment variable not set")
31     # Make a request to the external API using the API key
32     url = "https://api.example.com/data"
33     headers = {
34         "Authorization": f"Bearer {api_key}"
35     }
36     response = requests.get(url, headers=headers)
37     return response.json()
38 # Example usage
39 if __name__ == "__main__":
40     try:
41         data = get_api_data()
42         print("API data retrieved successfully!")
43         print(data)
44     except Exception as e:
45         print(f"Error: {e}")
46 # Note: In a real application, you would set the API_KEY environment variable securely and not hard-code it in your script.
47
48

```

Output :

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe
/OneDrive/Pictures/Desktop/AIAC/assg_20.py
API_KEY environment variable not set
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>

```

Task 3 – File Upload Security Validation

Task: Test an AI-generated file upload program for unsafe file handling. Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename Handling.

Expected Output:

- A secure file upload implementation.

```

47
48 #generate a Flask file upload example. Check for vulnerabilities like uploading executable files. Add validation for file type, size, and secure filename
49 from flask import Flask, request, redirect, url_for
50 from werkzeug.utils import secure_filename
51 import os
52 app = Flask(__name__)
53 # Configure upload folder and allowed extensions
54 UPLOAD_FOLDER = 'uploads/'
55 ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}
56 app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
57 def allowed_file(filename):
58     return '.' in filename and \
59         filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS
60 @app.route('/upload', methods=['GET', 'POST'])
61 def upload_file():
62     if request.method == 'POST':
63         # Check if the post request has the file part
64         if 'file' not in request.files:
65             return "No file part"
66         file = request.files['file']
67         # If user does not select file, browser also submit an empty part without filename
68         if file.filename == '':
69             return "No selected file"
70         if file and allowed_file(file.filename):
71             filename = secure_filename(file.filename)
72             file.save(os.path.join(app.config['UPLOAD_FOLDER'], filename))
73             return "File uploaded successfully!"
74         else:
75             return "Invalid file type. Allowed types are: png, jpg, jpeg, gif."
76     return '''
77     <doctype html>
78     <title>Upload a File</title>
79     <h1>Upload a File</h1>
80     <form method=post enctype=multipart/form-data>
81         <input type=file name=file>
82         <input type=submit value=Upload>
83     </form>
84     '''
85 if __name__ == '__main__':
86     # Ensure the upload folder exists
87     if not os.path.exists(UPLOAD_FOLDER):
88         os.makedirs(UPLOAD_FOLDER)
89     app.run(debug=True)
90

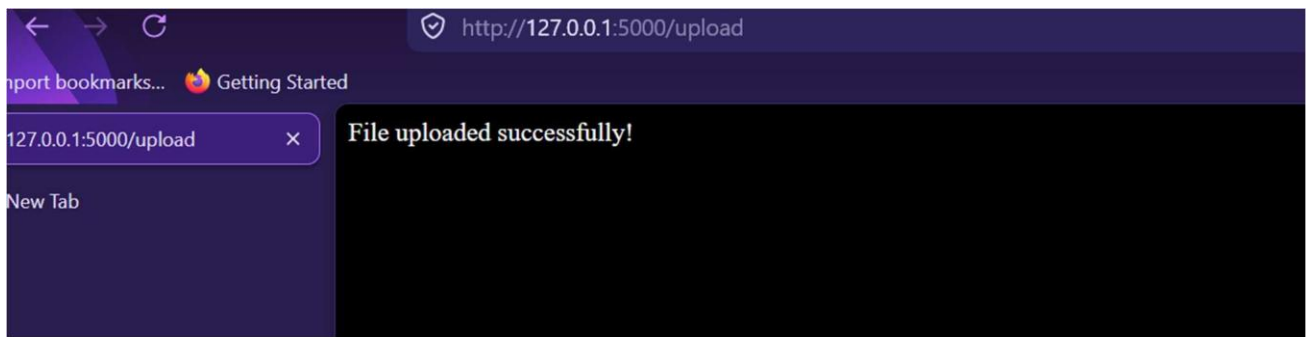
```

Output :

```

/OneDrive/Pictures/desktop/ATAC/assg_20.py
* Serving Flask app 'assg_20'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 765-492-300
127.0.0.1 - - [05/Apr/2026 10:51:49] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [05/Apr/2026 10:51:49] "GET /favicon.ico HTTP/1.1" 404 -

```



Task 4 – Command Injection Prevention

Task: Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using `os.system()` or `subprocess`.
- Identify command injection risks with user input.
- Fix using safe `subprocess` calls and input validation.

Expected Output:

- A secure command execution script.

```

92 #generate code using os.system() or subprocess. Identify command injection risks with user input. Fix using safe subprocess calls and input validation.
93 import subprocess
94 def run_command(user_input):
95     # Validate user input to prevent command injection
96     if not user_input.isalnum():
97         raise ValueError("Invalid input: only alphanumeric characters are allowed.")
98
99     # Use subprocess to run a safe command
100     result = subprocess.run(
101         f"echo {user_input}",
102         shell=True,
103         capture_output=True,
104         text=True
105     )
106     return result.stdout.strip()
107
108 # Example usage
109 if __name__ == "__main__":
110     user_input = input("Enter a string to echo: ")
111     try:
112         output = run_command(user_input)
113         print("Command output:", output)
114     except ValueError as e:
115         print(f"Error: {e}")
116

```

Output :

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/BHARATH/OneDrive/Pictures/Desktop/AIAC/assg_20.py
Enter a string to echo: hello
Command output: hello
PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC>

```

Task 5 – Secure Input Handling in Web Forms

Task: Check AI-generated web form code for improper input validation. Instructions:

- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.
- Fix using server-side validation and sanitization.

Expected Output:

- A secure form resistant to injection attacks.

```

116
117 #generate an HTML + Flask feedback form. Identify missing validation and unsafe rendering. Fix using server-side validation and sanitization.
118 from flask import Flask, request, render_template_string
119 import html
120 app = Flask(__name__)
121 @app.route('/feedback', methods=['GET', 'POST'])
122 def feedback():
123     if request.method == 'POST':
124         # Get user input and sanitize it
125         user_feedback = request.form.get('feedback', '')
126         sanitized_feedback = html.escape(user_feedback)
127
128         # Here you would typically save the feedback to a database
129         return f"Thank you for your feedback: {sanitized_feedback}"
130
131     # Render the feedback form
132     return '''
133     <doctype html>
134     <title>Feedback Form</title>
135     <h1>Feedback Form</h1>
136     <form method=post>
137         <textarea name=feedback rows=4 cols=50></textarea><br>
138         <input type=submit value=Submit>
139     </form>
140     '''
141 if __name__ == '__main__':
142     app.run(debug=True)
143
144

```

Output :

```

PS C:\Users\BHARATH\OneDrive\Pictures\Desktop\AIAC> & C:\Users\BHARATH\AppData\Local\Programs\Python\Python313\python.exe c:/Users/B
/OneDrive/Pictures/Desktop/AIAC/assg_20.py
* Serving Flask app 'assg_20'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 765-492-300

```

